Phase 5 — Signal Delivery & Return

KERNEL — deliver_signal()

Step 1: Save Original Context

```
user_esp = tf->esp;   // current user stack
saved_eip = tf->eip;  // where to resume later
saved_esp = tf->esp;  // stack to restore later
```

Step 2: Write Trampoline to User Stack

```
// At user_esp, write machine code:
B8 1C 00 00 00   // MOV EAX, 28 (SYS_SIGRETURN)
CD 30            // INT 0x30 (syscall)
EB FE            // JMP -2 (infinite loop safety)
```

7 bytes of executable code on the stack

Step 3: Push Signal Frame

```
push(trampoline_addr);  // fake return address
push(signum);           // handler argument
push(saved_esp);        // for sigreturn
push(saved_eip);        // for sigreturn
```

Stack layout after rewrite

Step 4: Rewrite Trap Frame

```
tf->eip = sa->sa_handler;  // jump to handler
tf->esp = new_esp;         // use modified stack
```

When `IRET` executes → CPU loads these values → **Execution redirected to user handler!**

User Stack After Rewrite

Address	Content	Purpose
ESP+28	original ESP	pre-signal stack top
ESP+24	saved EIP	resume address
ESP+20	saved ESP	stack to restore
ESP+16	signum (2 or 11)	handler argument
ESP+12	→ trampoline	fake return addr
ESP+4	`B8 1C 00 00 00`	MOV EAX, 28
ESP+0	`CD 30 EB FE`	INT 0x30; JMP -2

IRET with modified tf

═══ IRET — Ring 0 → Ring 3 ═══ CPU loads EIP, ESP, EFLAGS from trap frame ═══

CPU jumps to sa_handler(signum)

USER SPACE (Ring 3)

User Handler Executes

```
void my_handler(int signum) {
  printf("Caught signal %d\n", signum);
  // can call kill(), printf(), etc.
}
// When handler returns...
// RET pops return addr → trampoline!
```

Runs in **user mode** (Ring 3) Full access to user-space functions

RET pops trampoline addr

Trampoline Executes

Handler's `RET` instruction pops trampoline address from stack:

```
MOV EAX, 28      ; SYS_SIGRETURN
INT 0x30          ; trap into kernel
JMP -2            ; safety: never reached
```

→ Back into kernel for cleanup

INT 0x30 → kernel

sys_sigreturn()

Kernel reads saved values from user stack:

```
saved_eip = *(user_esp + 4);
saved_esp = *(user_esp + 8);
tf->eip = saved_eip;  // original EIP
tf->esp = saved_esp;  // original ESP
```

Context fully restored!

EIP/ESP restored

Process Resumes ✅

- Original EIP restored
- Original ESP restored
- Process continues exactly where it was interrupted
- **Completely transparent!**